Logbook Entry Generator

Setup Guide

PionCT Experiment — Jefferson Lab

June 24, 2026

Contents

1. Introduction

This guide is for experiment coordinators who need to configure and distribute the logbook tool for their experiment. It covers editing the configuration file, running the build script, and setting up Google Sheets export.

2. Prerequisites

- Python 3.6 or later (standard library only — no `pip` installs needed)
- A terminal (any operating system)
- Optionally: a Google account with access to Google Drive/Sheets, if you want the Sheets export feature

3. File Layout

After downloading, your working directory should look like this:

```
my_experiment/
+-- build.py          <- build script (do not edit)
+-- config.json       <- your experiment configuration (edit this)
+-- logo.png          <- your experiment logo (optional)
`-- output/           <- generated files appear here (created
   automatically)
    +-- logbook_generator.html
    `-- apps_script.gs   (only if Google Sheets is enabled)
```

Distribute `output/logbook_generator.html` to your shift workers. Everything they need is embedded in that single file — no other files need to travel with it. The logo is base64-embedded automatically if it is found next to `config.json` at build time.

4. Editing `config.json`

Open `config.json` in any text editor. The following sections describe each configurable field.

4.1 Experiment Identity

```
"experiment_name": "PionCT",
"page_title":      "PionCT Logbook Entry Generator",
"contact_email":   "gayoso@jlab.org",
"credit_line":     "Original idea from Andrew Schick for the
   PRadII experiment"
```

- `experiment_name` — used to namespace the browser's localStorage key so sessions from different experiments do not collide. Keep it short with no spaces.
- `page_title` — displayed as the H1 heading and browser tab title.
- `contact_email` — shown at the bottom of the Session Management box.
- `credit_line` — optional acknowledgement line. Remove the key entirely to omit it.

4.2 Logo

```
"logo": {
  "path": "logo.png",
  "alt":  "My Experiment Logo"
}
```

- `path` — relative to `config.json`. If the file exists there, the build script embeds it as base64 in the HTML (fully self-contained, no path dependency). If it does not exist, the path is used as-is (suitable for hosted URLs such as `https://...`).
- Set `"logo": {}` or remove the key entirely to show no logo.

4.3 Quick Links

```
"quick_links": [
  { "label": "Short Term Run Plan",           "id": "runPlanLink"
      },
  { "label": "Link to Shift Worker Checklist","id":
     "checklistLink" }
]
```

Each entry creates a URL input field in the Summary section. `label` is the display text; `id` must be a unique alphanumeric identifier (no spaces). Add, remove, or relabel as many as your experiment needs. The import feature uses the label text to identify links when re-parsing a generated HTML entry.

4.4 Google Sheets

```
"google_sheets": {
  "enabled":       true,
  "shared_secret": "CHANGE_ME_TO_SOMETHING_PRIVATE"
}
```

- Set `"enabled": false` to remove all Sheets UI from the generated HTML and skip generating `apps_script.gs`.
- `shared_secret` — any string you choose. It must match the value in the deployed Apps Script. Keep it private; it is the only access control on who can POST rows to your sheet.

4.5 Run Fields

This is the most important section. It defines every column in the run summary table and (if Sheets is enabled) the Google Sheet.

```
"run_fields": [
  {
    "key":          "run-number",
    "label":        "Run Number",
    "type":         "text",
    "source":       "user",
    "sheet_column": "Run Number",
    "sheet_align":  "right"
  },
  ...
]
```

Property	Required	Values	Meaning
`key`	yes	hyphenated	CSS class used internally. Must be unique.
`label`	yes	any string	Label shown in the form and table header.
`type`	yes	`text`, `textarea`	Input type. Use `textarea` for Comments.
`source`	yes	see below	Where the value comes from.
`sheet_column`	yes	any string	Column header written to Google Sheets.
`sheet_align`	yes	`left`, `right`, `center`	Alignment in the HTML table.
`placeholder`	no	any string	Placeholder text in the input.
`_comment`	no	any string	Ignored by build script. For your notes.

`source` values

- `"user"` — shown as an editable field in the form; sent to the sheet.
- `"auto"` — shown read-only in the form (greyed out); computed automatically. Two auto fields are currently supported:
  - Key `"date"` — filled with the local date (`YYYY-MM-DD`) at generate/send time. Not shown in the form at all.

  - Key `"run-length"` — auto-calculated as Stop − Start in minutes. Shown in the form but read-only.

- `"sheet-only"` — not shown in the HTML form at all. Written as an empty cell to the sheet (to be filled manually in the spreadsheet later).

> **Warning:** The two `auto` keys (`date` and `run-length`) have special hardcoded behaviour in the build script. Any other field with `source: "auto"` will appear as a greyed-out text input but will not be auto-populated.

Field order matters. The order of entries in `run_fields` determines the left-to-right column order in both the HTML table and the Google Sheet. See Chapter 6 if you change the order after deploying the Apps Script.

5. Running the Build Script

```
python3 build.py              # uses config.json in the current
    directory
python3 build.py my_exp.json # uses a specific config file
```

Output is written to `output/` next to the config file. The script will:

1. Validate `config.json` and exit with a clear error if anything is wrong.
2. Embed the logo as base64 if the file is found (prints a confirmation).
3. Write `output/logbook_generator.html`.
4. Write `output/apps_script.gs` (only if `google_sheets.enabled` is true).
5. Print the schema hash (see Chapter 6).

Rerun the script any time you change `config.json`. The output files are always regenerated from scratch.

6. Schema Hash and Changing Fields Mid-Experiment

The build script computes a hash of your `run_fields` column order and saves it to a hidden file `.schema_hash` next to `config.json`. On the next build, it compares the new hash to the saved one.

If the hash changes and Sheets is enabled, the script prints:

```
+------------------------------------------------------------------+
|  WARNING: SHEETS SCHEMA CHANGED                                  |
|  The run field schema has changed since the last build.          |
|  You MUST redeploy the Apps Script and update the Web App URL    |
|  in the logbook tool's Google Sheets Export Settings, or the     |
|  column order in your sheet will be wrong.                       |
+------------------------------------------------------------------+
```

What this means in practice: if you add, remove, or reorder columns in `run_fields`, the existing Apps Script deployment maps columns by position, not by name. A mismatch will silently write values into the wrong columns.

> **Note:** If `.schema_hash` is missing (e.g. it was deleted or this is the first build), the comparison is skipped silently and a new hash file is written. No data is lost.

The current schema hash is also embedded as a comment in the generated HTML and at the top of `apps_script.gs`, so you can always check which build produced a given file.

7. Setting Up Google Sheets Export

7.1 Step 1 — Create the Google Sheet

Create a new Google Sheet. The build script will write a header row automatically the first time a row is appended, but you can also add the header manually if you want to set column widths or formatting in advance. The column order must match the `sheet_column` values in `config.json` in the same order as `run_fields`.

7.2 Step 2 — Open Apps Script

In the Google Sheet: **Extensions** → **Apps Script**.

Delete any placeholder code in the editor. Paste the entire contents of `output/apps_script.gs`.

7.3 Step 3 — Set the Shared Secret

In the script, find this line near the top:

```
const SHARED_SECRET = 'CHANGE_ME_TO_SOMETHING_PRIVATE';
```

Replace the placeholder with whatever string you set in `config.json` under `google_sheets.shared_secret`. They must match exactly.

7.4 Step 4 — Deploy as Web App

Click **Deploy** → **New deployment**.

- Type: **Web app**
- Execute as: **Me**
- Who has access: **Anyone**

Click **Deploy**, authorise the permissions when prompted, then copy the **Web App URL** that appears. It looks like:
`https://script.google.com/macros/s/XXXXXXXXXX/exec`

> **Security note:** "Anyone" means anyone with the URL can POST to the script. The shared secret is the only access control. Keep the URL and secret out of public repositories.

7.5 Step 5 — Distribute to Shift Workers

Open `output/logbook_generator.html` in a browser. In the **Google Sheets Export Settings** section, paste:

- **Web App URL** — the URL from Step 4
- **Shared Secret** — the same string from your config

These are saved in the browser's localStorage automatically. Each shift worker on a different machine needs to do this once.

7.6 Redeploying After a Schema Change

If you change `run_fields` and the build script warns you about a schema change:

1. Rebuild to get the new `apps_script.gs`.
2. In Apps Script: **Deploy** → **Manage deployments** → **Edit (pencil icon)**.
3. Change "Version" to **New version**.
4. Click **Deploy**.
5. The URL **does not change** on redeployment — you do not need to update it in the logbook tool.

8. Distributing the Tool

Send shift workers `output/logbook_generator.html`. That is the only file they need. The logo and all CSS are embedded. They open it locally in a browser — no web server required.

If you want to host it (e.g. on a lab web server or GitHub Pages), that works too — just upload the single HTML file.